

CAP theorem evaluation on Redis: Availability, Consistency, and Partitioning

MASTER DEGREE IN COMPUTER SCIENCE

Big Data Analytics – Academic Year 2025/2026

Author:

Pellacani Simone (ID: 217424)

Repository:

https://github.com/s0SimoneP0s/BD_assignements/BD

13 luglio 2026

UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA



Sommario

Questo rapporto presenta una valutazione sperimentale di un sistema distribuito NoSQL basato su Redis. L'obiettivo è misurare il comportamento del sistema sotto guasti controllati e partizioni di rete, collegando le osservazioni al teorema CAP. Il codice dei test e gli script di orchestrazione sono disponibili nel repository del progetto e su https://github.com/s0SimoneP0s/BD_assignments.

Indice

Sommario	1
1 Requisiti	2
2 Obiettivi	2
3 Cosa testeremo: teorema CAP e modelli NoSQL	2
4 Redis data model e implicazioni per i test	2
4.1 Panoramica	2
4.2 Modelli di dati in Redis	2
5 Progettazione e architetture supportate	3
5.1 Replicazione, persistenza e implicazioni di consistenza	4
5.2 Atomicità	4
5.3 Cluster Sharding	4
5.4 Persistenza	4
6 Setup sperimentale e metodologia dei test	4
6.1 Test generico	4
7 API e workload	5
7.0.1 Dettagli dei test	5
8 Discussione e conclusioni	5
Riferimenti	6

1 Requisiti

- **Funzionali:** deploy Redis multinodo capace di accettare letture e scritture concorrenti; accesso al cluster tramite localhost per eseguire workload e iniettare guasti.
- **Non-funzionali:** scenari containerizzati e riproducibili; misurazione e log di tassi di successo; script per inizializzare e ripristinare gli esperimenti.

2 Obiettivi

- Misurare i compromessi tra disponibilità e consistenza durante partizioni se evidenziati.
- Fornire test ripetibili che simulino guasti e producano log leggibili dalla macchina.
- Riportare la correttezza (violazioni di consistenza) e metriche operative.

3 Cosa testeremo: teorema CAP e modelli NoSQL

Il **teorema CAP** afferma che un sistema dati distribuito può fornire al massimo due delle seguenti tre garanzie simultaneamente: Consistenza, Disponibilità e Tolleranza alle partizioni [1]. In pratica: [4]

- **Disponibilità** (availability): ogni richiesta a un nodo non fallito riceve una risposta (senza garantire sia il valore più recente). verificheremo se il sistema continua ad accettare e servire richieste corrette quando alcuni nodi falliscono.
- **Consistenza** (copy-consistency): tutti i nodi vedano gli stessi dati nello stesso momento. verificheremo se le repliche ritornano lo stesso stato dopo eventi casuali sui nodi di rete.
- **Tolleranza alle partizioni** (partitioning): il sistema continua a operare nonostante partizioni di rete arbitrarie. verificheremo se il sistema continua a operare anche quando le partizioni di rete vengono forzate.

Gli esperimenti sono condotti in un contesto ideale: un piccolo cluster containerizzato. I test sono progettati per misurare disponibilità, diverse nozioni di consistenza e comportamento di recupero.

4 Redis data model e implicazioni per i test

Redis è un sistema ricco di funzionalità, ma nel contesto di questo lavoro si presta a un uso semplice: è facile da deployare, configurare e integrare nel cluster di test. In particolare, useremo solo la parte più essenziale, il **String key-value**. Di seguito una panoramica dei tipi di dato principali, coerente con la documentazione ufficiale di Redis.

4.1 Panoramica

Redis è principalmente uno store key-value che espone diversi tipi di dato nativi [3]. Ogni tipo offre garanzie semantiche e pattern d'uso differenti; queste scelte influenzano replicazione, concorrenza e recupero. Abbiamo scelto appositamente questo strumento perché unisce un insieme molto ricco di funzionalità a un uso pratico semplice sia a livello implementativo, sia nella fase di deploy sia durante la configurazione del sistema.

4.2 Modelli di dati in Redis

L'obiettivo non è mostrare come anche il modello influenzi la correttezza dell'applicazione, ma guidare la progettazione dei test per verificarli adeguatamente in un caso base.

Idealmente il sistema gestisce separatamente le strutture dati dalla logica di partizionamento e replicazione, quindi verrà testato il comportamento di una struttura dati campione.

Qua a seguito una veloce panoramica dei principali tipi di dato in Redis, con esempi di utilizzo e pattern d'uso tipici utilizzando API python.

- **Stringhe:** sono la scelta corretta per valori atomici, contatori, cache di piccole dimensioni e flag di stato. Risolvono il problema del salvataggio semplice chiave-valore e supportano operazioni rapide e atomiche come aggiornamenti, letture e incrementi.

```
r.set("user:1:name", "Mario")
name = r.get("user:1:name")
r.incr("page:views")
```

- **Hash:** sono utili quando un'entità ha molti attributi, ad esempio un profilo utente, un record prodotto o una configurazione. Risolvono il problema di aggiornare solo alcuni campi senza riscrivere tutto l'oggetto.

```
r.hset("user:1", mapping={"name": "Mario", "age": 22})
user_name = r.hget("user:1", "name")
r.hincrby("user:1", "age", 1)
```

- **Liste:** sono adatte a code di lavoro, feed temporali, log append-only e buffer di messaggi. Risolvono il problema dell'ordinamento degli elementi e dell'elaborazione FIFO/LIFO.

```
r.lpush("jobs", "job-1")
next_job = r.rpop("jobs")
size = r.llen("jobs")
```

- **Set:** sono indicate per rappresentare collezioni di elementi unici, come tag, utenti online o insiemi di appartenenza. Risolvono il problema dei duplicati e rendono efficienti test di membership e operazioni insiemistiche.

```
r.sadd("online-users", "alice")
is_member = r.sismember("online-users", "alice")
members = r.smembers("online-users")
```

- **Sorted set (zset):** sono utili per classifiche, leaderboard, priorità e range di punteggio, ad esempio top utenti o job scheduling. Risolvono il problema dell'ordinamento dinamico con punteggi aggiornabili.

```
r.zadd("leaderboard", {"alice": 120, "bob": 95})
top = r.zrevrange("leaderboard", 0, 2, withscores=True)
score = r.zscore("leaderboard", "alice")
```

- **Stream:** sono adatti a pipeline di eventi, telemetria e processing distribuito con consumer group. Risolvono il problema dell'ingestione ordinata dei messaggi e della loro elaborazione affidabile.

```
msg_id = r.xadd("events", {"type": "login", "user": "alice"})
entries = r.xrange("events", count=1)
r.xgroup_create("events", "workers", id="0", mkstream=True)
```

- **Tipi specializzati:** bitmap, HyperLogLog e strutture geospaziali sono utili per casi come conteggi approssimati, tracciamento di flag compatti e query geografiche. Risolvono problemi di spazio, cardinalità approssimata e rappresentazione efficiente di dati derivati.

```
r.setbit("flags", 7, 1)
count = r.pfcount("unique-visitors")
r.geoadd("stores", {"store-1": (12.5, 41.9)})
```

Nel nostro esperimento, però, questi modelli non vengono esercitati tutti: il workload di test usa principalmente la struttura **String**, tramite operazioni SET/GET, perché l'obiettivo è misurare disponibilità, consistenza e tolleranza ai guasti del cluster nel caso più semplice e controllabile.

5 Progettazione e architetture supportate

Redis è un archivio dati in-memory che opera come database persistente tramite posix COW snapshot. La sua architettura è ottimizzata per garantire latenze estremamente ridotte, delegando la gestione della durabilità e della distribuzione a meccanismi configurabili.

5.1 Replicazione, persistenza e implicazioni di consistenza

In modalità cluster (a seconda della configurazione) le chiavi possono essere distribuite tra più nodi o accumulate nel primary e poi distribuite tra i master (sharded). Questo significa che, se un client deve lavorare su informazioni salvate su nodi diversi in configurazione sharding, le operazioni devono essere coordinate con attenzione oppure ricostruite lato client unendo i risultati ottenuti dai vari nodi.

Anche gli stream, cioè flussi ordinati di messaggi, e i consumer group, cioè gruppi di processi che leggono e gestiscono questi messaggi, introducono aspetti di ordinamento e distribuzione del carico. Per questo sono rilevanti quando si valuta se i messaggi vengono letti ed elaborati in modo coerente. Ciò dimostra che Redis non è solo un archivio semplice di coppie chiave-valore: quando i dati sono distribuiti su più nodi o quando entrano in gioco flussi di messaggi intra-cluster, tutt'altro che banali.

Nel test è stata usata una configurazione priva di sharding.

Conseguenze progettuali:

- Per un comportamento fortemente consistente, i client dovrebbero preferire letture dal primary oppure usare livelli supportati da consenso; in caso contrario, le letture dalle repliche possono essere "Soft state".
- Comandi atomici come HSET e LPUSH riducono alcuni problemi di concorrenza, ma non verranno trattati questi scenari. Per gestire carichi concorrenti il client python è stato gestito con un lock per serializzare le operazioni di scrittura e lettura.

5.2 Atomicità

Redis utilizza un'architettura **single-threaded** per l'elaborazione dei comandi. Questo semplifica la gestione della concorrenza: ogni comando viene eseguito in modo sequenziale e atomico, eliminando la necessità di lock complessi internamente al database e garantendo che le operazioni sulle strutture dati siano sempre consistenti a livello di singolo nodo.

5.3 Cluster Sharding

In configurazione Cluster Sharded, la distribuzione dei dati avviene tramite gli **hash slot**. Lo spazio delle chiavi è diviso in 16.384 slot distribuiti tra i nodi master tramite hash function. Questo meccanismo permette di scalare il sistema aggiungendo nodi e ridistribuendo gli slot in modo trasparente.

5.4 Persistenza

Per la durabilità dei dati, Redis offre due meccanismi:

- **RDB (Redis Database Backup)**: snapshot puntuali del dataset a intervalli definiti. È efficiente per i riavvii ma può comportare perdite di dati tra uno snapshot e l'altro.
- **AOF (Append-Only File)**: un log di tutte le operazioni di scrittura. Garantisce una persistenza maggiore (quasi real-time) a fronte di file di log più grandi. Soggetto attenzionato dal test di partizionamento.

6 Setup sperimentale e metodologia dei test

I workload e le procedure automatizzate di test tra cui la metodologia è stata presa come spunto [2] vengono eseguiti tramite orchestrazione containerizzata (Docker Compose) per garantire riproducibilità e facilità di implementazione. I generatori di carico sono script Python personalizzati; gli eseguibili nella cartella `experiments` eseguono letture e scritture secondo gli scenari descritti sotto.

6.1 Test generico

- T: numero di thread concorrenti simulati (thread/processi). Aumentando questo valore si incrementa la concorrenza totale.
- N: numero di operazioni locali eseguite da ciascun thread; il totale delle operazioni è $T \times N$.
- P: numero di nodi nel cluster Redis; 3 nel nostro caso.

Algorithm 1 Test generico

```
1: procedure TESTGENERICO( $P, T, N$ )
2:   Avviare cluster con  $P$  nodi
3:   Lanciare  $T$  thread worker
4:   for  $t = 1$  to  $T$  do
5:     Ogni worker esegue  $N$  operazioni locali e casuali:
6:     per ciascuna e registra (timestamp, operazione, risultato)
7:   end for
8:   Attendere il completamento dei worker e raccogliere i log
9: end procedure
```

7 API e workload

Sotto una breve descrizione dei test e delle chiamate API usate per generare carico e misurare il comportamento del cluster Redis. Non sempre si tratta di chiamate esposte direttamente dal client Redis ufficiale, ma a volte sono state progettate apposite funzioni wrapper che orchestrano le operazioni e registrano i log.

- **start_stop_container**: script o controlli sui container che arrestano/riavviano i nodi e manipolano le regole di rete, ad esempio con `tc`, per creare partizioni.
- **scan_all**: i client registrano lo stato del nodo, l'esito e il valore osservato per le letture.
- **random_rw_event**: esegue operazioni casuali di lettura e scrittura.
- **random_partition_event**: esegue eventi casuali di partizionamento della rete (spegne/riavvia nodi e legge/scrive dati nel frattempo).
- **random_primary_switch**: esegue eventi casuali di cambio dei nodi primari.

7.0.1 Dettagli dei test

I log grezzi relativi agli esperimenti sono raccolti nella cartella `experiments/logs/` del repository; il riepilogo delle esecuzioni batch è disponibile in `experiments/logs/stats.csv`. Per il repository dei log si può usare il percorso https://github.com/s0SimoneP0s/BD_assignments/BD/experiments/logs, dove viene mostrata solo la parte CSV.

- **Test di consistenza**: le esecuzioni con `test_consistency` verificano se il cluster mantiene un comportamento coerente sotto carico concorrente e durante i cambi di primario.
- **Test di disponibilità**: le esecuzioni con `test_availability` verificano se il cluster continua a rispondere alle operazioni di base anche in presenza di disturbi controllati.
- **Test di partizionamento**: le esecuzioni con `test_partitioning` osservano il comportamento del sistema quando la rete viene perturbata in modo controllato.

8 Discussione e conclusioni

Il file `stats.csv` riassume 36 esecuzioni del piano sperimentale, distribuite sui tre scenari `test_consistency`, `test_partitioning` e `test_availability`, con valori di `thread_count` pari a $\{1, 8, 32\}$ e `local_test_count` pari a $\{1, 8, 32, 128\}$. In tutte le run il campo `exit_code` vale 0 e `result` vale ok, quindi non emergono fallimenti operativi né anomalie esplicite nei log.

Nel complesso, i risultati confermano che il protocollo sperimentale è ripetibile e che il cluster Redis ha mantenuto un comportamento stabile in tutte le configurazioni testate. Questa stabilità è coerente con un setup containerizzato e ben orchestrato, ma non basta per una validazione generale del teorema CAP: lo studio mostra che, nel contesto osservato, i test sono stati eseguiti correttamente, non che il sistema garantisca sempre consistenza forte o disponibilità in presenza di partizioni.

In sintesi, il lavoro evidenzia la solidità del piano sperimentale e la sua ripetibilità, ma per trarre conclusioni più forti sarebbero necessari scenari di guasto più aggressivi, cluster più ampi e metriche aggiuntive sulla qualità delle risposte sotto partizione.

Riferimenti bibliografici

- [1] Brewer, E. A. (2000). Towards robust distributed systems (cap theorem). Keynote, PODC conference. Available at: <https://people.eecs.berkeley.edu/~brewer/cs262b-2004/PODC-keynote.pdf>.
- [2] Jepsen (2026). Distributed systems testing. Accessed 2026-06-19.
- [3] Redis Labs (2024). Redis documentation. Accessed 2026-06-19.
- [4] Wikipedia (2026). Teorema cap — wikipedia, l'enciclopedia libera. [Online; in data 19-giugno-2026].